Name: Shiom Shrivastava
Enroll: 23100BTCSAII14323

EXPERIMENT – 1

❖ AIM:

To examine list reports, group data, include list headers and footers, format list columns, enhance a list report, understand fact and measure data, understand aggregate data, understand difference in aggregation, explore data aggregation, use shared dimensions to create multi-fact queries, create a multi-fact query in a list report, add repeated information to reports, create a mailing list report, focus reports using filters, create filters, filter data with advanced detail filters, apply filters with aggregation, apply detail filters on fact data in a report, filter data using summary filters, apply summary filter to a report, apply pre-defined source filters, and create a report focused on top performing product types and product lines.

❖ THEORY:

This experiment demonstrates the creation and analysis of a Sales Data Warehouse using a Star Schema model. The database consists of one Fact table (sales_fact) and three Dimension tables (product_dim, customer_dim, and date_dim).

The sales_fact table stores transactional data such as quantity and sales amount, while the dimension tables store descriptive information related to products, customers, and time.

Using SQL queries, data is retrieved by joining fact and dimension tables to generate detailed list reports. Aggregation functions such as SUM() and GROUP BY are used to calculate product-wise and month-wise sales. The experiment also demonstrates the use of:

- Detail filters using WHERE clause
- Summary filters using HAVING clause
- Product type and product line analysis
- Header and footer generation for report formatting
- Pre-defined filters for state-based sales analysis

Name: Shiom Shrivastava
Enroll: 23100BTCSAII14323

❖ PROCEDURE:

Step 1: Create Database

```
mysql> CREATE DATABASE company;
Query OK, 1 row affected (0.00 sec)
```

```
mysql> use company;
Database changed
```

Explanation:

- CREATE DATABASE company; creates a new database named company.
- USE company; selects this database so that all tables and data will be created inside it.

Step 2: Create Tables

```
mysql> CREATE TABLE product_dim (
    ->     product_id INT PRIMARY KEY,
    ->     product_name VARCHAR(50),
    ->     product_type VARCHAR(30),
    ->     product_line VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql> CREATE TABLE customer_dim (
    ->     customer_id INT PRIMARY KEY,
    ->     customer_name VARCHAR(50),
    ->     city VARCHAR(30),
    ->     state VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.03 sec)

mysql> CREATE TABLE date_dim (
    ->     date_id INT PRIMARY KEY,
    ->     year INT,
    ->     month VARCHAR(15)
    -> );
Query OK, 0 rows affected (0.04 sec)

mysql> CREATE TABLE sales_fact (
    ->     sale_id INT PRIMARY KEY,
    ->     date_id INT,
    ->     product_id INT,
    ->     customer_id INT,
    ->     quantity INT,
    ->     sales_amount INT
    -> );
Query OK, 0 rows affected (0.03 sec)
```

Explanation:

In this step, four tables are created in the database:

1. **product_dim** – Stores product details such as product ID, product name, product type, and product line.

Name: Shiom Shrivastava
Enroll: 23100BTCSAII14323

2. **customer_dim** – Stores customer information like customer ID, name, city, and state.
3. **date_dim** – Stores time-related data such as date ID, year, and month.
4. **sales_fact** – Stores sales transaction data including sale ID, date ID, product ID, customer ID, quantity sold, and sales amount.

The first three tables are **dimension tables** because they store descriptive information.
The **sales_fact** table is the **fact table** because it stores measurable data like quantity and sales amount.

Step 3: Insert Product Data

```
mysql> INSERT INTO product_dim VALUES
    -> (1,'Helmet','Safety','Mining'),
    -> (2,'Boots','Safety','Mining'),
    -> (3,'Gas Sensor','Safety','Mining'),
    -> (4,'Drill Machine','Equipment','Mining'),
    -> (5,'Oxygen Mask','Safety','Mining');
Query OK, 5 rows affected (0.01 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Explanation:
This step inserts product records such as Helmet, Boots, Gas Sensor, Drill Machine, and Oxygen Mask into the product dimension table.

Step 4: Insert Customer Data

```
mysql> INSERT INTO customer_dim VALUES
    -> (1,'Amit','Dhanbad','Jharkhand'),
    -> (2,'Rahul','Ranchi','Jharkhand'),
    -> (3,'Suresh','Bokaro','Jharkhand'),
    -> (4,'Ravi','Asansol','West Bengal'),
    -> (5,'Neha','Durgapur','West Bengal'),
    -> (6,'Pooja','Patna','Bihar'),
    -> (7,'Rohit','Gaya','Bihar'),
    -> (8,'Karan','Nagpur','Maharashtra'),
    -> (9,'Anjali','Raipur','Chhattisgarh'),
    -> (10,'Vikas','Bilaspur','Chhattisgarh');
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

Name: Shiom Shrivastava
Enroll: 23100BTCSAII14323

Explanation:
This step inserts 10 customer records with different cities and states.
This allows sales analysis based on customer location

Step 5: Insert Date Data

```
mysql> INSERT INTO date_dim VALUES
    -> (1,2024,'Jan'),
    -> (2,2024,'Feb'),
    -> (3,2024,'Mar');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Explanation:
This step inserts month and year data (Jan, Feb, Mar 2024).
This helps in performing time-based aggregation like monthly sales reports

Step 6: Insert Sales Data

```
mysql> INSERT INTO sales_fact VALUES
    -> (1,1,1,1,5,5000),
    -> (2,1,2,2,10,12000),
    -> (3,1,3,3,3,15000),
    -> (4,1,4,4,1,50000),
    -> (5,2,5,5,4,8000),
    -> (6,2,1,6,6,6000),
    -> (7,2,2,7,8,10000),
    -> (8,2,3,8,2,9000),
    -> (9,3,4,9,1,52000),
    -> (10,3,5,10,5,10000);
Query OK, 10 rows affected (0.01 sec)
Records: 10  Duplicates: 0  Warnings: 0
```

Explanation:
This step inserts sales transaction data.
Each record connects:

- A product
- A customer
- A date
- Quantity sold
- Sales amount

This table links all dimension tables and forms the core of analysis.

Name: Shiom Shrivastava
Enroll: 23100BTCSAII14323

Step 7: List Reports

```
mysql> SELECT
    ->     c.customer_name,
    ->     p.product_name,
    ->     d.month,
    ->     s.quantity,
    ->     s.sales_amount
    -> FROM sales_fact s
    -> JOIN customer_dim c ON s.customer_id = c.customer_id
    -> JOIN product_dim p ON s.product_id = p.product_id
    -> JOIN date_dim d ON s.date_id = d.date_id;
+---------------+---------------+-------+----------+--------------+
| customer_name | product_name  | month | quantity | sales_amount |
+---------------+---------------+-------+----------+--------------+
| Amit          | Helmet        | Jan   |        5 |         5000 |
| Rahul         | Boots         | Jan   |       10 |        12000 |
| Suresh        | Gas Sensor    | Jan   |        3 |        15000 |
| Ravi          | Drill Machine | Jan   |        1 |        50000 |
| Neha          | Oxygen Mask   | Feb   |        4 |         8000 |
| Pooja         | Helmet        | Feb   |        6 |         6000 |
| Rohit         | Boots         | Feb   |        8 |        10000 |
| Karan         | Gas Sensor    | Feb   |        2 |         9000 |
| Anjali        | Drill Machine | Mar   |        1 |        52000 |
| Vikas         | Oxygen Mask   | Mar   |        5 |        10000 |
+---------------+---------------+-------+----------+--------------+
10 rows in set (0.00 sec)
```

Explanation:
This query joins the fact table with all dimension tables to generate a complete report showing:

- Customer Name
- Product Name
- Month
- Quantity
- Sales Amount

This is called a **detailed list report**.

Step 8: GROUP DATA (Product wise sales)

```
mysql> SELECT
    ->     p.product_name,
    ->     SUM(s.sales_amount) AS total_sales
    -> FROM sales_fact s
    -> JOIN product_dim p ON s.product_id = p.product_id
    -> GROUP BY p.product_name;
+---------------+-------------+
| product_name  | total_sales |
+---------------+-------------+
| Boots         |       22000 |
| Drill Machine |      102000 |
| Gas Sensor    |       24000 |
| Helmet        |       11000 |
| Oxygen Mask   |       18000 |
+---------------+-------------+
5 rows in set (0.02 sec)
```

Name: Shiom Shrivastava
Enroll: 23100BTCSAII14323

Explanation:
GROUP BY p.product_name;
This query calculates total sales for each product using:
SUM(s.sales_amount)
It performs aggregation and shows product-wise total revenue.

Step 9: Fact & Measure Calculation

```
mysql> SELECT
    ->     SUM(quantity) AS total_units,
    ->     SUM(sales_amount) AS total_sales
    -> FROM sales_fact;
+-------------+-------------+
| total_units | total_sales |
+-------------+-------------+
|          45 |      177000 |
+-------------+-------------+
1 row in set (0.00 sec)
```

Explanation:

- quantity and sales_amount are **measures**.
- This query calculates:
  - Total units sold
  - Total sales revenue

It demonstrates understanding of fact and measure data.

Step 10: Month-wise Aggregation

```
mysql> SELECT
    ->     d.month,
    ->     SUM(s.sales_amount) AS monthly_sales
    -> FROM sales_fact s
    -> JOIN date_dim d ON s.date_id = d.date_id
    -> GROUP BY d.month;
+-------+---------------+
| month | monthly_sales |
+-------+---------------+
| Feb   |         33000 |
| Jan   |         82000 |
| Mar   |         62000 |
+-------+---------------+
3 rows in set (0.00 sec)
```

Name: Shiom Shrivastava
Enroll: 23100BTCSAII14323

Explanation:
This query calculates total sales for each month.
It shows how aggregation works based on time dimension.

Step 11: Detail Filter (High sales records)

```
mysql> SELECT *
    -> FROM sales_fact
    -> WHERE sales_amount > 10000;
+---------+---------+------------+-------------+----------+--------------+
| sale_id | date_id | product_id | customer_id | quantity | sales_amount |
+---------+---------+------------+-------------+----------+--------------+
|       2 |       1 |          2 |           2 |       10 |        12000 |
|       3 |       1 |          3 |           3 |        3 |        15000 |
|       4 |       1 |          4 |           4 |        1 |        50000 |
|       9 |       3 |          4 |           9 |        1 |        52000 |
+---------+---------+------------+-------------+----------+--------------+
4 rows in set (0.00 sec)
```

Explanation:
This filter shows only those sales records where sales amount is greater than 10,000.
This is called a **detail filter** because it filters individual rows before grouping.

Step 12: Summary Filter (Group-level Filter)

```
mysql> SELECT
    ->     p.product_name,
    ->     SUM(s.sales_amount) AS total_sales
    -> FROM sales_fact s
    -> JOIN product_dim p ON s.product_id = p.product_id
    -> GROUP BY p.product_name
    -> HAVING SUM(s.sales_amount) > 15000;
+--------------+-------------+
| product_name | total_sales |
+--------------+-------------+
| Boots        |       22000 |
| Drill Machine |     102000 |
| Gas Sensor   |       24000 |
| Oxygen Mask  |       18000 |
+--------------+-------------+
4 rows in set (0.00 sec)
```

Explanation:
This filter is applied after grouping.
It shows only those products whose total sales exceed 15,000.
This is called a **summary filter**.

Name: Shiom Shrivastava
Enroll: 23100BTCSAII14323

Step 13: Top Product Types

```
mysql> SELECT
    ->     p.product_type,
    ->     SUM(s.sales_amount) AS total_sales
    -> FROM sales_fact s
    -> JOIN product_dim p ON s.product_id = p.product_id
    -> GROUP BY p.product_type
    -> ORDER BY total_sales DESC;
+--------------+-------------+
| product_type | total_sales |
+--------------+-------------+
| Equipment    |      102000 |
| Safety       |       75000 |
+--------------+-------------+
2 rows in set (0.00 sec)
```

Explanation:
This query:

- Groups data by product type (Safety, Equipment)
- Calculates total sales
- Orders results in descending order

It helps identify top-performing product categories.

Step 14: Top Product Lines

```
mysql> SELECT
    ->     p.product_line,
    ->     SUM(s.sales_amount) AS total_sales
    -> FROM sales_fact s
    -> JOIN product_dim p ON s.product_id = p.product_id
    -> GROUP BY p.product_line;
+--------------+-------------+
| product_line | total_sales |
+--------------+-------------+
| Mining       |      177000 |
+--------------+-------------+
1 row in set (0.00 sec)
```

Explanation:
This query calculates total sales based on product line (Mining).
It helps in category-level performance analysis.

Step 15: List Header & Footer

Name: Shiom Shrivastava
Enroll: 23100BTCSAII14323

```
mysql> SELECT
    ->    'MINING SALES REPORT' AS Report_Title,
    ->    c.customer_name,
    ->    p.product_name,
    ->    s.sales_amount
    -> FROM sales_fact s
    -> JOIN customer_dim c ON s.customer_id = c.customer_id
    -> JOIN product_dim p ON s.product_id = p.product_id;
+--------------------+---------------+---------------+--------------+
| Report_Title       | customer_name | product_name  | sales_amount |
+--------------------+---------------+---------------+--------------+
| MINING SALES REPORT | Amit          | Helmet        |         5000 |
| MINING SALES REPORT | Pooja         | Helmet        |         6000 |
| MINING SALES REPORT | Rahul         | Boots         |        12000 |
| MINING SALES REPORT | Rohit         | Boots         |        10000 |
| MINING SALES REPORT | Suresh        | Gas Sensor    |        15000 |
| MINING SALES REPORT | Karan         | Gas Sensor    |         9000 |
| MINING SALES REPORT | Ravi          | Drill Machine |        50000 |
| MINING SALES REPORT | Anjali        | Drill Machine |        52000 |
| MINING SALES REPORT | Neha          | Oxygen Mask   |         8000 |
| MINING SALES REPORT | Vikas         | Oxygen Mask   |        10000 |
+--------------------+---------------+---------------+--------------+
10 rows in set (0.05 sec)
```

```
mysql> SELECT
    ->    'TOTAL' AS Footer,
    ->    SUM(sales_amount) AS Grand_Total
    -> FROM sales_fact;
+--------+-------------+
| Footer | Grand_Total |
+--------+-------------+
| TOTAL  |      177000 |
+--------+-------------+
1 row in set (0.00 sec)
```

In this step, a header and footer are added to the report.

Header:
The query adds a fixed title **"MINING SALES REPORT"** using:
'MINING SALES REPORT' AS Report_Title
This title appears in every row of the report along with customer name, product name, and sales amount. It improves the presentation of the report.

Footer:
The second query calculates the overall total sales using:
SUM(sales_amount) AS Grand_Total
It displays **TOTAL = 177000**, which represents the grand total of all sales.

Name: Shiom Shrivastava
Enroll: 23100BTCSAII14323

Step 16: Pre-defined Filter (Rational Rose style)

```
mysql> SELECT
    ->     c.customer_name,
    ->     p.product_name,
    ->     s.sales_amount
    -> FROM sales_fact s
    -> JOIN customer_dim c ON s.customer_id = c.customer_id
    -> JOIN product_dim p ON s.product_id = p.product_id
    -> WHERE c.state = 'Jharkhand';
+---------------+--------------+--------------+
| customer_name | product_name | sales_amount |
+---------------+--------------+--------------+
| Amit          | Helmet       |         5000 |
| Rahul         | Boots        |        12000 |
| Suresh        | Gas Sensor   |        15000 |
+---------------+--------------+--------------+
3 rows in set (0.00 sec)
```

Explanation:
This query filters the report to show only customers from Jharkhand.
It demonstrates how to apply a predefined source filter for region-based analysis.

❖ **RESULT:**

The Sales Data Warehouse was successfully created. Reports were generated using joins, aggregation, and filters. Product and month-wise sales analysis was performed successfully.

Name: Shiom Shrivastava
Enroll: 23100BTCSAII14323

EXPERIMENT – 2

❖ AIM:

To create a Crosstab report, add measures to the Crosstab report, use data sources for Crosstab, create simple and complex Crosstab reports, create Crosstab nodes and node members, format the Crosstab report, and sort the Crosstab report using SQL queries.

❖ THEORY:

A **Crosstab Report** is used to display summarized data in a **matrix format**, where:

- **Rows** represent one field (example: Product)
- **Columns** represent another field (example: Region)
- **Measures** represent calculated values like **SUM**, **COUNT**, etc.

Crosstab reports are very useful in business intelligence and reporting tools because they help in:

- Easy comparison of data
- Analyzing trends across categories
- Presenting large data in a compact form

In SQL, Crosstab reports are created using **GROUP BY**, **CASE statements**, and **aggregate functions** like **SUM()** and **COUNT()**.

Name: Shiom Shrivastava
Enroll: 23100BTCSAII14323

❖ PROCEDURE:

Step 1: Create Data Source for Crosstab

```
mysql> CREATE TABLE Sales (
    ->     SaleID INT PRIMARY KEY,
    ->     Product VARCHAR(50),
    ->     Region VARCHAR(50),
    ->     Amount INT
    -> );
Query OK, 0 rows affected (0.11 sec)
```

Query: Create Table
Explanation:

- This query creates a table named **Sales**.
- SaleID uniquely identifies each sale.
- Product stores product name.
- Region stores sales region.
- Amount stores sales value.

```
mysql> INSERT INTO Sales VALUES
    -> (1, 'Laptop', 'North', 50000),
    -> (2, 'Laptop', 'South', 45000),
    -> (3, 'Mobile', 'North', 20000),
    -> (4, 'Mobile', 'South', 22000),
    -> (5, 'Tablet', 'North', 15000),
    -> (6, 'Tablet', 'South', 17000);
Query OK, 6 rows affected (0.02 sec)
Records: 6  Duplicates: 0  Warnings: 0
```

Query: Insert Data
Explanation:

- This query inserts sales records into the **Sales** table.
- Each product has sales data for **North** and **South** regions.

Name: Shiom Shrivastava
Enroll: 23100BTCSAII14323

Step 2: Create a Simple Crosstab Report

```
mysql> SELECT
    ->     Product,
    ->     SUM(CASE WHEN Region = 'North' THEN Amount ELSE 0 END) AS North,
    ->     SUM(CASE WHEN Region = 'South' THEN Amount ELSE 0 END) AS South
    -> FROM Sales
    -> GROUP BY Product;
+---------+-------+-------+
| Product | North | South |
+---------+-------+-------+
| Laptop  | 50000 | 45000 |
| Mobile  | 20000 | 22000 |
| Tablet  | 15000 | 17000 |
+---------+-------+-------+
3 rows in set (0.00 sec)
```

Explanation:

- CASE WHEN separates sales based on region.
- SUM() calculates total sales for each region.
- GROUP BY Product groups data product-wise.
- Products appear as **rows**, regions as **columns**

Step 3: Add Measures to Crosstab Report

```
mysql> SELECT
    ->     Product,
    ->     SUM(CASE WHEN Region = 'North' THEN Amount ELSE 0 END) AS North_Total,
    ->     COUNT(CASE WHEN Region = 'North' THEN 1 END) AS North_Count,
    ->     SUM(CASE WHEN Region = 'South' THEN Amount ELSE 0 END) AS South_Total,
    ->     COUNT(CASE WHEN Region = 'South' THEN 1 END) AS South_Count
    -> FROM Sales
    -> GROUP BY Product;
+---------+-------------+-------------+-------------+-------------+
| Product | North_Total | North_Count | South_Total | South_Count |
+---------+-------------+-------------+-------------+-------------+
| Laptop  |       50000 |           1 |       45000 |           1 |
| Mobile  |       20000 |           1 |       22000 |           1 |
| Tablet  |       15000 |           1 |       17000 |           1 |
+---------+-------------+-------------+-------------+-------------+
3 rows in set (0.00 sec)
```

Explanation:

- Adds **multiple measures**:
  - SUM() → total sales
  - COUNT() → number of sales entries
- Separate measures are calculated for **North** and **South**

Name: Shiom Shrivastava
Enroll: 23100BTCSAII14323

Step 4: Create Complex Crosstab Report

```
mysql> SELECT
    ->     Product,
    ->     SUM(CASE WHEN Region = 'North' THEN Amount ELSE 0 END) AS North,
    ->     SUM(CASE WHEN Region = 'South' THEN Amount ELSE 0 END) AS South,
    ->     SUM(Amount) AS Total_Sales
    -> FROM Sales
    -> GROUP BY Product;
+---------+-------+-------+-------------+
| Product | North | South | Total_Sales |
+---------+-------+-------+-------------+
| Laptop  | 50000 | 45000 |       95000 |
| Mobile  | 20000 | 22000 |       42000 |
| Tablet  | 15000 | 17000 |       32000 |
+---------+-------+-------+-------------+
3 rows in set (0.00 sec)
```

Explanation:

- This query adds **Total_Sales**.
- Combines regional sales with overall sales.
- Makes the Crosstab report more detailed.

Step 5: Create Crosstab Nodes and Node Members

```
mysql> SELECT DISTINCT Region FROM Sales;
+--------+
| Region |
+--------+
| North  |
| South  |
+--------+
2 rows in set (0.00 sec)
```

Explanation:

- Fetches unique regions from the table.
- These regions act as **Crosstab column node members**.
- Helps define dynamic Crosstab columns.

Name: Shiom Shrivastava
Enroll: 23100BTCSAII14323

Step 6: Format Crosstab Report

```
mysql> SELECT
    ->     Product AS Product_Name,
    ->     SUM(CASE WHEN Region = 'North' THEN Amount ELSE 0 END) AS North_Sales,
    ->     SUM(CASE WHEN Region = 'South' THEN Amount ELSE 0 END) AS South_Sales
    -> FROM Sales
    -> GROUP BY Product;
+--------------+-------------+-------------+
| Product_Name | North_Sales | South_Sales |
+--------------+-------------+-------------+
| Laptop       |       50000 |       45000 |
| Mobile       |       20000 |       22000 |
| Tablet       |       15000 |       17000 |
+--------------+-------------+-------------+
3 rows in set (0.00 sec)
```

Explanation:

- Uses **alias names** to improve readability.
- Makes column names user-friendly.
- Used for report presentation.

Step 7: Sort and Format Crosstab Report

```
mysql> SELECT
    ->     Product,
    ->     SUM(CASE WHEN Region = 'North' THEN Amount ELSE 0 END) AS North,
    ->     SUM(CASE WHEN Region = 'South' THEN Amount ELSE 0 END) AS South,
    ->     SUM(Amount) AS Total_Sales
    -> FROM Sales
    -> GROUP BY Product
    -> ORDER BY Total_Sales DESC;
+---------+-------+-------+-------------+
| Product | North | South | Total_Sales |
+---------+-------+-------+-------------+
| Laptop  | 50000 | 45000 |       95000 |
| Mobile  | 20000 | 22000 |       42000 |
| Tablet  | 15000 | 17000 |       32000 |
+---------+-------+-------+-------------+
3 rows in set (0.00 sec)
```

Explanation:

- ORDER BY Total_Sales DESC sorts data in **descending order**.
- Products with highest sales appear first.
- Improves analysis and decision-making.

❖ RESULT:

In this experiment, Crosstab reports were successfully created using SQL. Simple and complex Crosstabs were designed, measures were added, nodes and node members were defined, and sorting and formatting were applied to present data clearly and effectively.